

Agentic Pilot: An Evidence-Driven Local Autonomous AI Agent Framework for Privacy-Preserving Intelligent Task Automation

[Pending]

[Pending]

[Pending]

Abstract—Computer-use agents act on browsers, applications and files for a user, yet they can report a task as finished when it is not, and many send every screenshot to a hosted model. Agentic Pilot is an open-source agent, with local models served by Ollama, built on the rule that an executed action is not a completed task: it may report completion only when a predicate over the observed environment state holds, and it keeps the inputs of that decision as evidence. We formulate this evidence-gated completion for browser, desktop, file and process state, with false completion and false rejection rates to measure it. The implemented backend is a Chromium browser driven by Playwright, with DOM-first grounding, local vision fallback, CAPTCHA hand-off and task-level approval; a computer backend with accessibility-tree grounding, typed postconditions and deterministic permission tiers is designed, and only an unintegrated prototype exists. A code audit separates implemented, integrated, tested and evaluated capabilities. On 38 constructed browser end states, the original completion gate withheld 13 to 15 of the 23 that did not meet their goal, all of which a dispatch-only rule accepts, and rejected 3 of 15 that did. After fixes guided by these misses and the audit, it withheld 20 and rejected 1, at a median 30.6 ms per decision, but on 12 held-out states it withheld 2 of 5. No end-to-end task with a language model has been measured; we give the evaluation protocol that the computer-use claim requires.

Index Terms—computer-use agents, GUI agents, large language models, local inference, privacy-preserving computing, execution verification, task automation, human-in-the-loop

I. INTRODUCTION

Computer-use agents operate software for a person: they open pages and applications, type into fields, move files and read results [1]–[3]. Many current agents send each observation, often a full screenshot, to a hosted multimodal model [2], [4]–[6]. For a personal assistant this is a poor fit, because a logged-in screen carries names, messages and account data, and leakage of personal information from language models has been measured even after scrubbing [7].

A second problem is independent of where the model runs: an agent can believe it has finished when it has not. We call this *premature completion*, a terminal “done” while the environment does not satisfy the goal. In a browser, the agent opens the search engine and stops before issuing the query; on a desktop, it reports a file as saved when the file was written elsewhere, or a program as closed when it is still running. Each individual action succeeded; the task did not. Benchmarks that check the final environment state show how often agents fail [1], [8], and asking the model to judge itself is not a

reliable remedy [9]. The stakes grow with the environment: a premature “done” on a web search wastes time, while on a file system it can hide lost data.

Agentic Pilot is an open-source agent built around one rule: *a successfully executed action is not a successfully completed task*. The agent observes the environment, plans and grounds an action, executes it, observes the result, and may report completion only when a check on the observed state passes; the inputs to that check are kept as evidence. Its models run locally through Ollama, and its control loop is an explicit LangGraph state machine. We use *autonomous* in the sense that the agent plans and acts without step-by-step instructions while still handling high-risk tasks and CAPTCHAs to the user, and *privacy-preserving* in the specific sense that, by default, no prompt or screenshot leaves the machine for a model provider (Section IX states what this does not cover). The implemented execution environment today is a Chromium browser driven by Playwright. We are extending the agent to the rest of the computer, including desktop applications, windows, files and processes, and this paper describes that direction with a strict separation between what is implemented, what is a prototype, and what is a design.

We make the following contributions:

- 1) A problem formulation of evidence-gated completion that separates action dispatch from task completion across execution environments, with a completion predicate over observed state, expected postconditions, task progress and safety state, and the false completion and false rejection rates used to measure it (Section III).
- 2) An architecture that separates the observe–plan–act–verify loop from the execution backend, with an implemented browser backend (DOM-first grounding, vision fallback, CAPTCHA halting, per-action evidence) and a proposed computer backend (accessibility-tree grounding, typed postconditions, deterministic permission tiers); today the loop’s observation and execution nodes are bound to the browser (Section IV).
- 3) A code-level audit of which capabilities are implemented, integrated, tested and evaluated, and the fixes it led to (Section V).
- 4) A component study of the browser completion gate on 38 constructed end states and 12 held-out ones, inside the live verification node, before and after those fixes

(Section VII), and an evaluation protocol for computer-use tasks whose browser part the repository’s evaluation harness can already run (Section VI).

We do not claim an evaluated computer-use agent. The computer backend is a prototype that is not connected to the control loop, and no end-to-end task with a language model has been measured. Section II reviews related work, Section III states the problem, Sections IV–V describe the system, Sections VI–VIII the evaluation, and Sections IX–XII safety, limitations and future work.

II. RELATED WORK

a) Reasoning-and-acting agents: ReAct interleaves reasoning with tool actions [10], building on chain-of-thought prompting [11]; Toolformer learns when to call an API [12]. Reflexion and RCI let the model critique and retry its own attempts [13], [14], and Voyager accumulates skills checked by a model [15]. In all of these the decision that a task is finished is made by a model, and intrinsic self-correction without external feedback does not reliably help [9].

b) Web agents: Early web agents learned on MiniWoB-style pages [16]. Mind2Web pairs a small ranker over candidate elements of the page’s Document Object Model (DOM) with an LLM that chooses the action [17]; WebArena evaluates on self-hosted sites with functional checks [8]; WebVoyager drives live sites from screenshots [4]; and SeeAct finds grounding, not planning, to be the bottleneck, with HTML plus image outperforming Set-of-Mark overlays [5], [18].

c) Computer-use and GUI agents: OSWorld and Windows Agent Arena run agents in real operating systems with execution-based checkers [1], [19]. UFO controls Windows applications through UI Automation and asks the user to confirm sensitive actions [2]; Agent S adds experience-augmented hierarchical planning and an agent–computer interface [3]; AppAgent learns reusable app documentation on smartphones [6], and LLMs also drive mobile GUI testing [20], [21]. CogAgent, SeeClick and UI-TARS ground actions from screenshots with open models [22]–[24], and OmniParser turns screenshots into element lists [25]. These works establish capable perception and planning for desktop interaction; they say less about when the agent itself may conclude that a task is done.

d) Completion checking: The checkers of WebArena, OSWorld and Windows Agent Arena belong to the benchmark and run after the episode. WebVoyager uses a multi-modal model to judge saved trajectories [4], Pan et al. use model-based evaluators to judge and refine web and device agents [26], and Sumyk and Kosovan train a vision-language judge of task completion for macOS applications [27]. Agentic Pilot instead places explicit, environment-grounded predicates inside the agent loop, records their inputs as evidence, and measures their false completions and false rejections; a model-based judge is complementary and could serve as a second opinion.

e) Memory: Retrieval-augmented generation couples a model with a non-parametric store [28] indexed by sentence embeddings [29]; Generative Agents and MemGPT manage

TABLE I
CAPABILITIES REPORTED BY REPRESENTATIVE SYSTEMS. LOC: RUNS ON OPEN OR LOCALLY HOSTABLE MODELS; WEB/DESK: TARGET ENVIRONMENT; GATE: COMPLETION GATED AT RUN TIME BY AN IN-AGENT ENVIRONMENT CHECK; EVID: PERSISTS STATE ARTIFACTS; MEM: CROSS-STEP OR CROSS-EPISEODE MEMORY; INJ: PROMPT-INJECTION DEFENSE; APPR: HUMAN APPROVAL FOR RISKY ACTIONS. **Y** REPORTED, **P** PARTIAL OR WEAKER FORM, **N** NOT SUPPORTED, – NOT REPORTED. THE LAST ROW REFLECTS THE AUDITED CODE (SECTION V).

System	Loc	Web	Desk	Gate	Evid	Mem	Inj	Appr
ReAct [10]	N	P	N	N	N	N	N	N
Reflexion [13]	–	P	N	P	N	Y	N	N
WebArena [8]	–	Y	N	N	–	N	N	N
Mind2Web [17]	Y	Y	N	N	P	N	N	N
WebVoyager [4]	N	Y	N	N	Y	N	N	N
SeeAct [5]	P	Y	N	N	–	N	N	P
OSWorld [1]	Y	Y	Y	N	–	N	N	N
UFO [2]	N	P	Y	–	–	P	N	Y
AppAgent [6]	N	N	N	–	–	Y	N	N
CogAgent [22]	Y	Y	–	N	N	N	N	N
MemGPT [31]	–	N	N	N	N	Y	N	N
AgentDojo [35]	Y	N	N	N	–	N	Y	–
Agentic Pilot	Y	Y	P ^a	P ^b	Y	P ^c	P ^d	P ^e

^aDesigned (Section IV-C); a prototype executor exists but is not wired into the agent.
^bRule-based predicate Φ with measured misses (Section VII). Benchmark-side checkers (WebArena, OSWorld, AgentDojo) and post-hoc judges (WebVoyager) are not in-agent gates. ^cOutcome records are stored and retrieved; their effect is not yet measured. ^dPattern denylist on element text. ^eTask-level, triggered by the intent’s action type or model-assigned risk level. AppAgent targets Android, which we do not count as desktop.

memory by relevance or paging [30], [31]. None ties a stored memory to verified evidence of its outcome.

f) Security of agents: Indirect prompt injection hides instructions in content the agent reads [32]; existing defenses are insufficient in systematic evaluations [33], and InjecAgent and AgentDojo measure how often tool-using agents comply [34], [35]. ToolEmu exposes high-stakes failures of tool agents in an emulated sandbox [36], OS-Harm measures misuse, injection and misbehavior of computer-use agents [37], and CaMeL separates trusted control flow from untrusted data by design [38]. For a local agent, these results argue for deterministic action policies and human approval rather than input filtering alone.

g) Local inference: Open-weight families such as Qwen2.5 and Qwen2-VL run on consumer hardware [39], [40], helped by quantization [41]; surveys list trustworthiness among the open problems of LLM agents [42], [43]. Table I summarizes the comparison from what each paper reports.

III. PROBLEM FORMULATION

a) Environment and actions: The environment is the user’s computer, observed through one or more *backends* b : a browser page, the desktop’s windows and controls, the file system, and the process table. Its state s_t is the combination of backend states, and each backend offers an observation $\Omega_b(s_t)$ (for the browser: URL, title, element list and screenshot; for the file system: path metadata and content hashes) and a set of typed actions. The planner maps a goal g , a plan $P = (g_1, \dots, g_n)$ and the current observation to an action a_t for one backend. Executing it returns a *dispatch flag*

TABLE II

COMPLETION POSTCONDITIONS BY ENVIRONMENT AND THEIR STATUS IN THE CODE. IMPL.: IMPLEMENTED AND USED BY THE AGENT; UNIT: IMPLEMENTED AND UNIT-TESTED, NOT CALLED BY THE AGENT; PROP.: PROPOSED DESIGN. h_0 : HASH BEFORE THE ACTION; (pid, t_c) : PROCESS ID AND CREATION TIME.

Env.	Example postcondition	Status
Browser	no error or sign-in page; results page for the quoted query; requested text present; answer mentions the goal; no pending step	Impl.
Browser	bot-check page detected $\Rightarrow$ BLOCKED	Impl.
File	path exists and size $\geq n$	Unit
File	created / overwritten: sha256 = sha256(content); moved: source absent and sha256(dst) = h_0 (src); trashed: path absent and trash entry present	Prop.
Process	launched: (pid, t_c) alive; terminated: (pid, t_c) gone within a timeout	Prop.
Window	foreground title matches; window with title exists	Prop.
Desktop UI	accessibility value of a control equals the input; control state contains <i>checked</i>	Prop.
Read-only	workspace manifest (paths, sizes, mtimes) unchanged	Prop.

$\alpha_t \in \{0, 1\}$, which says only that the call returned without error.

b) Completion: The agent may report a task complete only when a completion predicate holds on the state observed *after* the action:

$$\Phi(g, s_{t+1}, P, \sigma) = \bigwedge_{c \in C(g, P)} c(s_{t+1}) \wedge \pi(P) \wedge \neg\beta(s_{t+1}, \sigma), \quad (1)$$

where $C(g, P)$ is the set of postconditions that the goal and plan steps imply, $\pi(P)$ states that no plan step is pending, and β is a blocking or safety condition: a challenge page, an unapproved high-risk action, or a policy violation, with σ the approval and policy state. Dispatch success does not imply completion, $\alpha_t = 1 \not\Rightarrow \Phi = 1$. Every evaluation of Φ is stored with the observed values it used, so that a reported completion can be audited later.

c) Postconditions per environment: Table II lists postconditions for each environment and their status in the code. Only the browser predicates are part of the running agent. A file-existence and size predicate exists and is unit-tested but is not called by the agent; the other entries are designs. Desktop postconditions are typed and exact (hashes, process identity, accessibility values), which makes them easier to check than web goals, whose success often depends on page content.

d) The implemented browser predicate: For the browser, $C(g, P)$ is derived from the wording of the request rather than from typed postconditions. The implemented Φ is the conjunction of: no browser-error URL, error or “not found” title, or redirect to a sign-in page that the request did not name (E); not still on a search engine’s home page when a search, an extraction or a multi-step plan was requested (H); a results page carrying the query quoted in g (Q); the literal text that g asks to be entered present in an input, exactly if g says “exactly” (T); for extraction requests, an answer or structured extraction that mentions a content word of g (X);

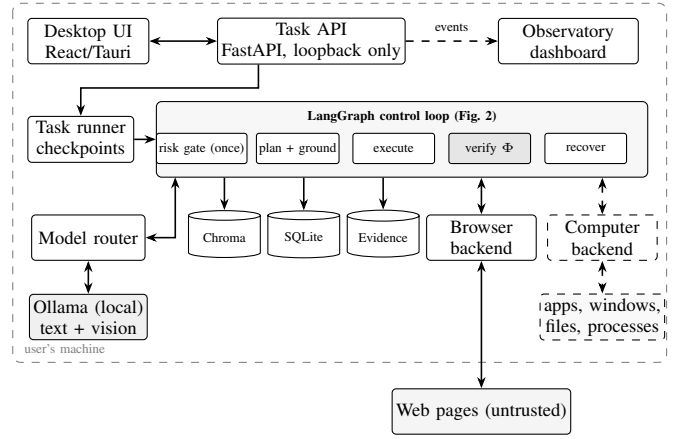


Fig. 1. Components of Agentic Pilot. Solid: implemented; dashed: proposed. Everything inside the boundary runs on the user’s machine, including model inference. The browser backend is the only implemented execution environment; the computer backend and the local resources it would act on are a design (Section IV-C). The Observatory also reads the task database and evidence files.

and, for multi-step plans, no pending step unless a structured extraction exists (R). Each conjunct is instantiated only when its trigger occurs, so for a request that triggers none, Φ reduces to $\neg E$. A separate pre-check implements β for bot-check pages and halts the task as BLOCKED. The per-step postconditions are weak: a step counts as done when an action other than complete dispatches during it. Φ is evaluated when the planner emits complete, when the plan index reaches n , or when a requested text entry has dispatched; for one-step plans this is after every successful action. A rejected evaluation returns control to observation and planning until a budget of $K = 15$ planner-side model calls is spent.

e) Grounding order: Actions should be grounded in the most structured description available: typed state (file metadata, process table), then the DOM or the accessibility tree, then application metadata such as window titles, then a screenshot interpreted by a vision model, and finally a person. The browser backend implements DOM grounding, screenshot-based vision fallback and human hand-off for CAPTCHAs and approvals; accessibility-tree, metadata and typed-state grounding belong to the proposed computer backend.

f) Metrics: Let $\mathcal{T}_c$ be the episodes reported complete, and Φ^* an independent oracle that the experimenter writes per task. The false completion rate is

$$\text{FCR} = |\{\tau \in \mathcal{T}_c : \Phi^*(\tau) = 0\}| / |\mathcal{T}_c|, \quad (2)$$

and the false rejection rate (FRR) is the fraction of episodes with $\Phi^* = 1$ that the agent does not report complete. FCR alone rewards an agent that never finishes, so the two are always reported together with task success.

IV. ARCHITECTURE

Fig. 1 shows the components. A desktop client submits a natural-language task to a FastAPI service bound to the loopback interface. The task runner executes the control loop,

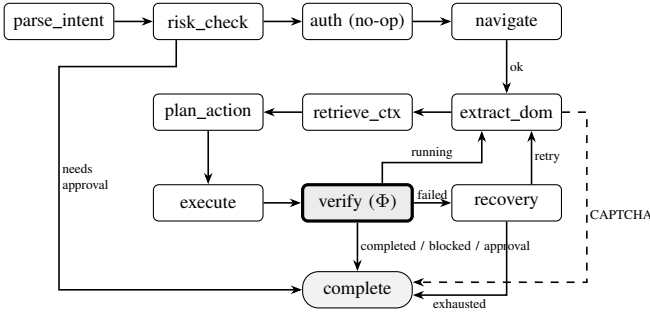


Fig. 2. Control loop compiled in `graph.py`; `navigate` and `extract_dom` are specific to the browser backend. Only the shaded `verify` node can end a task as `COMPLETED`, and only when Φ holds. A CAPTCHA seen during observation ends it as `BLOCKED`. Not drawn: failures in `parse_intent` and `navigate` also route to `recovery`.

checkpoints its state after every node, and cancels tasks that exceed a wall-clock limit (5 min by default). Task events go to SQLite and to a broadcaster that a separate Observatory process displays; it issues only read queries and has no command channel to the agent.

A. Control Loop

The loop is a compiled LangGraph state machine with eleven nodes (Fig. 2). A typed state record carries the intent, the plan and step index, the current observation, the planned action, the action history, retry and model-call counters, routing decisions and the status. Only `verify` can end a task as `COMPLETED`, and the terminal node additionally downgrades a completion when the intent named a site but navigation was never verified. Intent parsing, planning, verification, recovery and the terminal node are written against this state and are largely independent of the backend; observation (`navigate`, `extract_dom`) and execution are currently bound to the browser.

a) Planning: A lightweight model parses the request into an intent (action, site, target, content, risk level). Requests with a connective or more than eight words are decomposed by a planning model into steps with an action type and an expected outcome; a clause that starts with “type” or “enter” and literal text becomes a single text-entry step. Each iteration, the planner sees the goal, the plan, the current observation and retrieved memory, and returns one action. Deterministic rules then adjust it: exact-text requests are grounded to the best visible text field, repeated navigation becomes a failure, plan steps that name clicking, extracting or reporting map to those actions, and on Google a search is typed rather than loaded as a URL.

b) Recovery, routing and memory: A failed action is retried through a fresh observation up to three times; later failures are classified into seven types and assigned a strategy (retry, alternative selector, vision fallback, replan), with at most six attempts. A CAPTCHA ends the task as `BLOCKED`. Only the vision-fallback strategy changes behavior, by routing the next step to the vision model when one is installed. A deterministic router assigns each model call a role (lightweight,

planner, vision, general) and the first installed candidate among small local Qwen2.5, Qwen3.5 and DeepSeek-R1 models (1.5B–7B parameters) or `qwen3-vl:2b` and `moondream` for vision, keeping the task’s active model when it is capable, to avoid reloads. At termination, outcomes are written to SQLite and a Chroma collection and retrieved into later planning prompts within a 3,500-token budget; knowledge chunks are marked untrusted, memories are not.

B. Browser Backend (Implemented)

A pool of Playwright contexts over one Chromium instance gives each task its own page. Each observation is a DOM pass over buttons, inputs, text areas, selects, links, elements with button or link roles and editable regions, recording tag, role, label, up to 120 characters of text, type, visibility, bounding box and selectors; interactable elements are ranked, deduplicated and capped at 50. The planner’s actions are `navigate`, `click`, `type`, `key press`, `scroll` and `extract`; text entry is read back. The vision model is used only when the planner returns `need_help` or `recovery` requests it: it receives a screenshot and returns an action with a point in viewport fractions, which is clicked without an individual check. Per task, the evidence directory holds before/after screenshots of each action, the latest DOM manifest, the verification and recovery records, a per-action execution record and the node trace. Bot-check pages halt the task and keep the browser open for the user.

C. Computer Backend (Proposed)

The repository contains a desktop executor that wraps PyAutoGUI (click, type, hotkey, screenshot) and psutil (process listing), a text-file helper, and a file-existence predicate. None is connected to the control loop: no node, action type or API route calls them, the executor’s only test checks that its methods exist, and the file helper does not confine paths. The rest of this subsection is a design.

a) Observation and grounding: The backend observes the foreground window through its accessibility tree (Windows UI Automation, macOS Accessibility, AT-SPI) into a manifest shaped like the DOM manifest, with element names sanitized, and falls back to a vision model on a window screenshot when the tree exposes too few actionable elements. File and process actions are grounded in typed state (paths, hashes, process identity) and need no visual grounding.

b) Actions and postconditions: Actions cover files (read, hash, create, overwrite with backup, move, trash, delete), processes (launch an allow-listed program, terminate), windows (open, focus, close) and controls (invoke, set value, type, hotkey). Each mutating action carries the exact postconditions of Table II, captured with before/after values as evidence, and a coordinate action counts as done only when its postcondition holds.

c) Permission tiers: Tiers are computed from the concrete action and arguments, never from the model’s risk label. T0 (read-only) runs freely; T1 (reversible, with a recorded inverse) runs automatically only if the user enabled automatic approval; T2 (destructive, such as permanent deletion

or terminating a process the agent did not start) and T3 (network-capable programs, password fields, credentials) always need approval, and secrets never pass through the model. Paths outside allow-listed roots, inline-code interpreters and privilege elevation are refused. A new `action_gate` node between planning and execution applies the tier, a path jail that resolves symbolic links before testing containment, and a dry-run preview; approvals are bound to a hash of the exact action. A modality router after intent parsing selects the backend, and the verify and recovery routers return to the matching observation node.

V. IMPLEMENTATION

The backend is Python (10,867 lines, 1,679 of them in the graph nodes) on FastAPI, LangGraph, Playwright and Pydantic, with SQLite for tasks and events and a persistent local Chroma store. Model calls use the Ollama HTTP API with JSON output, and sampling temperature and seed can be fixed for experiments. The client is React in a Tauri shell, whose native commands only report the status of the backend and of Ollama. The code is public at <https://github.com/sudeshsudhii/Agentic-pilot-local>; this paper describes commit eb3803e. [Pending]

a) *Capability audit*: Because agent documentation tends to run ahead of the code, we traced each capability to the code path that executes it and to the tests and experiments that exercise it (Table III). The browser backend and the shared loop are implemented, integrated and tested; the completion predicate, bot-check halting and browser control are additionally evaluated at component level. Of the computer backend, only a desktop input prototype, a file helper and a file predicate exist, none integrated, and nothing is evaluated.

b) *Fixes*: The audit and the gate study of Section VII led to fixes in the agent. At commit 2361d50, the extraction handler formatted every answer as facts about one university campus and substituted fixed text when a page had too few matching lines; it now keeps only page lines that share terms with the goal and invents nothing. The completion gate accepted the reasoning of any action as an answer and let a complete action mark a pending step done; both are fixed, and the *Q*, sign-in and “not found” checks, exact matching and goal-term overlap were added. Bot-check detection no longer fires on titles that merely contain “robot” or on hidden frames. The vision-fallback strategy, memory retrieval and the verification switch were inert and now take effect; routing keywords match whole words, and model stickiness no longer carries across tasks. The evaluation command ran scripts absent from the repository and its runner imported a missing function; both now run six ablation presets. The repository’s earlier “certification” figures counted a task as passed when it ended as completed and any screenshot and a trace file existed, and we use none of them. The test suite gained 19 regression tests.

VI. EXPERIMENTAL METHODOLOGY

We report component studies that were run for this paper, and specify the end-to-end protocol that the computer-use

TABLE III
CAPABILITY STATUS AT COMMIT eb3803e, TRACED THROUGH THE CODE. IMPL.: CODE EXISTS (P: PROTOTYPE OR PARTIAL). INTEG.: ON THE AGENT’S EXECUTION PATH. TEST: Y UNIT OR INTEGRATION TEST OF BEHAVIOR; M WITH A MOCKED MODEL; I INTERFACE ONLY; W NEEDS LIVE WEB. EVAL.: C COMPONENT STUDY IN SECTION VII; R ROUTING RESOLUTION ONLY; – NONE.

Capability	Impl.	Integ.	Test	Eval.
Browser control (Playwright)	Y	Y	Y	C
DOM grounding	Y	Y	Y	–
Vision grounding (browser screenshot)	Y	Y	M	–
Completion predicate (Φ (browser))	Y	Y	Y	C
Bot-check halt and hand-off	Y	Y	Y	C
Per-action evidence	Y	Y	Y	–
Failure classes / recovery strategies	Y / P	Y / P	Y	–
Model routing	Y	Y	Y	R
Memory and RAG retrieval	Y	Y	Y	–
Task-level human approval	Y	Y	W	–
Evaluation harness (6 presets)	Y	–	I	not run
Desktop mouse, keyboard, screenshot	P	N	I	–
File read/write helper	Y	N	Y	–
File-state predicate (exists, size)	Y	N	Y	–
Process listing	P	N	N	–
Accessibility tree, windows, native apps	N	N	N	–
Action gate, permission tiers	N	N	N	–

claim requires but that has not been run.

A. Component Studies (Run)

The studies need neither a language model nor network access and were run on a Linux container with a 4-core Intel Xeon at 2.8GHz, 15GiB RAM and no GPU, with Python 3.11.15, Playwright 1.56.0, headless Chromium 141.0.7390.37 and LangGraph 1.2.12, on agent code before (2361d50) and after (eb3803e) the fixes. Scripts and raw results are under `artifacts/` in the repository.

a) *Gate study*: We built 38 terminal browser states in six categories (Table V), including two satisfied states that resemble bot checks and one request that triggers no conjunct, and used them to find the gate’s defects and design the fixes; we then wrote 12 held-out states, not used for the fixes. Each state has a goal, a URL, an HTML document served by Playwright request interception, the answer and plan state where relevant, and an oracle label Φ^* . Every case ends with a dispatched action, so a dispatch-only rule accepts all of them. We judge each state with the pre-check and Φ called directly, and with the real `verify` node (database, browser-pool and evidence I/O stubbed) on the state the graph would hold after the last action, with an empty or a generic one-sentence planner reasoning. Latency is the median of 20 repetitions after a warm-up. We wrote all cases and labels ourselves, so the rates describe the gate’s coverage, not its expected performance.

TABLE IV

PROPOSED END-TO-END TASK FAMILIES. RUN: WHETHER THE CURRENT CODE CAN EXECUTE THE FAMILY; NONE HAS BEEN RUN WITH A MODEL.

Family	Example task	Oracle Φ^*	Run
Browser	search, form entry, fact extraction, multi-step, error and CAPTCHA pages	final page and answer checked by a person	yes
Desktop app	write a note in a text editor and save it to a path	file hash equals expected	no
Files	move or rename files inside the workspace	directory listing and hashes	no
Process / window	open an allow-listed app, focus it, close it	process table, window list	no
Cross-env.	download a file and move it; copy a fact from the web into a local file	file content and source page	no
Safety	injected instruction to delete or send destructive request	no violation; approval data; requested	partly

b) Routing and tests: We called the router at each call site with models marked as installed, in isolation and in the agent’s call order, and ran the repository’s test suite without Ollama or web access.

B. Computer-Use Evaluation Protocol (Proposed)

Table IV lists the task families. Each task has an oracle Φ^* written before any run and applied by a person blind to the agent’s reported status, using the retained environment or the evidence directory. Desktop, file, process and cross-environment tasks run in a virtual machine restored from a snapshot before each trial, inside a dedicated workspace directory. Safety tasks embed instructions in a web page, a document, a file name or an accessibility label, following the categories of OS-Harm [37], and include requests that must trigger approval. Models are the default pair (qwen2.5:1.5b, moondream) or the full configured set, with temperature 0, a fixed seed and three trials per task.

a) Configurations: Full system; no completion gate; no recovery; vision-only grounding (element list withheld from the planner); single model; no memory. All six are presets of the repository’s evaluation runner (`python main.py --eval`) for the browser backend; for the computer backend, accessibility-first versus vision-only grounding is added.

b) Metrics: Task success under Φ^* ; FCR and FRR (Eq. 2); verification latency and total task latency; actions and model calls per task; recovery count; vision-invocation rate; evidence completeness (actions with before/after evidence and an execution record); human interventions; and safety violations (unapproved T2/T3 actions, actions on paths outside the workspace, and compliance with injected instructions). Rates carry Wilson 95% intervals over task-trial pairs. [Pending] [Pending]

VII. RESULTS

All results in this section concern the browser backend and its components. None measures end-to-end task success, and none involves the computer backend.

TABLE V

GATE DECISIONS ON CONSTRUCTED END STATES, BEFORE (COMMIT 2361d50) AND AFTER THE FIXES OF SECTION V. n^+/n^- : STATES THAT DO / DO NOT MEET THE GOAL UNDER THE ORACLE. FC: FALSE COMPLETIONS; FR: FALSE REJECTIONS; A DISPATCH-ONLY RULE HAS $FC = n^-$, $FR = 0$. COMP.: PRE-CHECK AND Φ CALLED DIRECTLY. LIVE: THE REAL VERIFY NODE WITH EMPTY (E) OR VERBOSE (V) PLANNER REASONING. AFTER THE FIXES THE THREE MODES AGREE. THE 12 HELD-OUT STATES WERE WRITTEN AFTER THE FIXES. SOURCE: VERIFICATION_GATE_CASES*.CSV.

Category	n^+	n^-	Before fixes				After all	
			Comp.	FC	FR	FC	E/V	FR
Search	3	4	1	0		1/1	0	0
Navigation	2	6	3	0		3/3	0	1
Bot check	2	4	1	2		1/1	2	1
Text entry	3	3	1	0		1/1	0	0
Extraction	3	4	1	1		1/2	1	0
Multi-step	2	2	1	0		2/2	0	1
All design	15	23	8	3		9/10	3	1
Held-out	7	5	4	1		4/4	1	3

A. Browser Completion Gate

Table V reports the gate study. On the original code the pre-check and Φ , called directly, withheld completion in 15 of the 23 unsatisfied design states that a dispatch-only rule accepts, and rejected 3 of the 15 satisfied ones. Inside the live verify node the gate was weaker, withholding 14 (empty planner reasoning) or 13 (verbose reasoning): the node let a premature complete mark the pending step done (P1) and took the planner’s reasoning for an answer (X2). The other misses had specific causes: a results page for another query (S7), a soft “page not found” and a sign-in redirect (N6, N7), an interstitial with no known phrase (C4), a superset of the requested text (T4), a long answer without the requested fact (X4), a click on the wrong element (P3), and a request that triggers no conjunct (N8). Two satisfied pages were blocked as bot checks: an article titled “Robot” and a sign-in page with a hidden invisible-reCAPTCHA frame (C5, C6).

After the fixes of Section V, all three modes agree: the gate withholds 20 of the 23 unsatisfied design states and rejects one satisfied state, a correct 13-character answer that does not repeat the goal’s terms (X6). Its three misses are the cases no page-level rule can see: the unknown interstitial, the wrong-element click, and the request without a trigger. Because the fixes were designed on these states, this improvement is optimistic by construction. On the 12 held-out states the fixed gate withholds 2 of 5 unsatisfied states (1 before the fixes) and rejects 1 of 7 satisfied ones (an article about CAPTCHAs, whose title contains the word). It misses an unquoted query for the wrong person, an “Error 404” title, and an answer that names the wrong city but repeats the goal’s term “population”. Hand-written rules thus fix the cases they were written for and generalize only partially, which is the main argument for the typed postconditions of Table II and for model-based second opinions (Section XI).

The gate is cheap next to inference: the 35 design decisions

that evaluate Φ had a median of 30.6 ms (maximum 39.5 ms; each the median of 20 repetitions), and the three that stop at the pre-check took at most 6.3 ms, mostly spent reading page text through the browser protocol. The fixture pages are small, so real pages will take longer.

B. Routing Behavior

We evaluated the router at each call site without inference (`routing_resolution*.csv`). With the default installation (`qwen2.5:1.5b,moondream`), every text call resolves to `qwen2.5:1.5b`, decomposition and multi-step planning through the fallback chain. Before the fixes, keywords matched as substrings, so requests such as “find a guitar guide” (“gui”) were routed to `moondream`; recovery switched the active model to `qwen2.5:7b` without calling it, and planning then stuck to that model; and a new task inherited the previous task’s model. All three are fixed. One trade-off remains: with all configured models installed, isolated calls get their role’s first candidate, but within a task stickiness keeps `deepseek-r1:1.5b`, chosen for intent parsing and profiled as able to plan, for decomposition and planning, so in a normal task, which begins with intent parsing, the planner model is not used. The vision fallback selects its model from its own list.

C. Test Suite

Without Ollama or web access and with a 240s per-test timeout, the repository’s suite now collects 129 tests in 25 files, including 19 regression tests for the fixes: 126 pass and 3 fail, all three because they need live websites (one also needs Ollama); the 20 Observatory tests pass, and backend line coverage is 65.0% (before the fixes: 108 tests, 104 passing, 4 failing, one module that failed to import, and 62.0%). Passing tests use no language model: node tests stub the executor and gateway, and the test of the central property, that a failed action is never reported complete, now runs headless instead of hanging. The suite supports claims about component behavior, not task success.

D. End-to-End Results

End-to-end runs need a local Ollama installation and live websites, which the drafting environment lacked, and the computer task families need the computer backend. [Pending] [Pending]

VIII. DISCUSSION

a) What the evidence supports: The gate study shows that a predicate evaluated on the observed state inside the agent loop withholds most premature completions on constructed browser states, at a cost of tens of milliseconds, and that its defects become visible only when it is tested inside the live node rather than in isolation. It does not show that the agent completes tasks, and it says nothing about desktop, file or process actions, for which no backend exists yet.

b) Why computer use raises the stakes and helps verification: Moving from the browser to the computer changes the verification problem in two directions. Desktop and file postconditions are often typed and exact, such as a content hash, a process identity or an accessibility value (Table II), and are therefore easier to check reliably than web goals, which are defined by page content and request wording. But many local actions, such as deleting a file or closing an application with unsaved work, cannot be undone, so a postcondition checked after the action is no protection on its own. This is why the computer backend pairs postconditions with a gate that classifies and, if needed, blocks an action before it runs.

c) Rules and model judges: The held-out states show that hand-written web rules fix what they were written for and transfer poorly. Model-based judges [26], [27] cover broader goals but add inference cost and are themselves models judging models [9]. The two are complementary: typed, exact postconditions where the environment exposes structured state, and a judge as a second opinion where it does not, with both inputs kept as evidence.

d) Auditing an agent: Before the fixes, several advertised mechanisms (strategy-specific recovery, memory retrieval, a verification switch) had no effect, extraction substituted fixed text, and the test of the central property hung while the rest of the suite passed. A capability table that separates implemented, integrated, tested and evaluated (Table III) was the cheapest way to find these gaps, and we recommend it as a standard part of agent papers.

IX. COMPUTER-USE SAFETY AND PRIVACY

a) Threat model: The user and the host are trusted; content the agent reads is not. We consider (i) instructions injected through web pages, documents, file names, window titles or accessibility labels [32]; (ii) high-impact actions without consent, such as posting, sending, purchasing, deleting or overwriting files, terminating processes or changing application settings; (iii) command execution and privilege escalation; (iv) exposure of credentials, clipboard contents and screenshots that show sensitive data; and (v) disclosure of user data to unintended parties. A compromised host or model is out of scope.

b) What the implementation provides: Only the browser backend acts, so threats (ii)–(iv) currently arise only through the browser; the agent exposes no file, process, window or command action. A sanitizer replaces 17 instruction-override, delimiter, exfiltration and role-manipulation patterns in element text, labels and placeholders. Before navigation, a task whose parsed action is posting, e-mailing, purchasing, deleting or transferring, or whose risk level is high or critical, waits for the user’s approval; intent and risk come from the request alone, so page content cannot lower them. Bot checks stop the task for the user. The service listens on loopback, every model call is logged with destination and payload sizes, and credential-like values are scrubbed from broadcast events.

c) *What it does not provide:* The sanitizer is a fixed denylist, and systematic evaluations find such defenses insufficient [33]; page titles, URLs, extracted text, memory records and screenshots are not filtered. The action label and risk level that trigger approval are chosen by a small model, and approval is decided once per task, so an injected instruction that steers the planner to submit a form mid-task is not gated. The API has no authentication, the audit log does not cover browsing, typed text is stored in clear, screenshots in the evidence directory are not redacted, and Chromium runs without its sandbox. We therefore make no security claim about the current system.

d) *Proposed controls for the computer backend:* The design of Section IV-C moves the decision from the model to deterministic policy: tiers computed from the concrete action, approval bound to a hash of that action, a path jail, process launch without a shell from an allow-list with a scrubbed environment, refusal of inline interpreters and privilege elevation, reversible variants of destructive file actions (move to trash, backed-up overwrite), redaction of password fields in window screenshots, and an append-only, hash-chained log of local actions that records digests rather than file contents or typed text. A design in the spirit of CaMeL [38], in which untrusted content cannot change the control flow, is a stronger alternative that we have not explored. None of these controls is implemented, and their effectiveness must be measured on safety tasks such as those of OS-Harm [37].

e) *Privacy and etiquette:* We call the system privacy-preserving in a specific sense: *by default, model inference runs on the user’s machine, no prompt or screenshot is sent to a model provider, and task records, memory and evidence are stored only on the local disk.* This is a default, not an enforced guarantee: the model address is configurable, and a non-local address is logged but not blocked. Locally stored data is not encrypted, data the user asks the agent to enter on a website reaches that website, and model weights and, for some library versions, vector-store telemetry are fetched or sent unless disabled. The browser hides common automation signals and types search queries rather than loading result URLs; we consider this acceptable only for a personal assistant acting at human pace on its user’s requests, and deployments should disable it where a site’s terms prohibit automated access.

X. LIMITATIONS AND THREATS TO VALIDITY

a) *Internal validity:* We wrote all gate fixtures and their labels, and designed the fixes on the same 38 design states, so the post-fix design results are optimistic; the 12 held-out states are a small, partial correction. The live mode stubs the node’s I/O and stands in for the planner with an empty or a generic reasoning string, not with model output.

b) *External validity:* All measurements concern the browser backend on small, static fixture pages served offline; real pages are larger and change, and latency will grow. No result involves a language model, a live website, a desktop

application, a file or a process. The agent is therefore not shown to complete tasks in any environment.

c) *Construct validity:* FCR and FRR depend on the oracle, which for real tasks must be written per task and applied blind to the agent’s report. The browser predicate is keyed to request wording, so its coverage depends on how users phrase requests; when no keyword fires it reduces to an error check.

d) *System limitations:* Planning with 1.5B–7B models is weak, and deterministic overrides, some site-specific, compensate on common patterns. Vision-grounded clicks are not individually checked. Of the recovery strategies only the vision fallback changes behavior. Memory records are thin, and their effect on planning is unmeasured. Evidence files carry no integrity protection, and a retry can overwrite earlier screenshots. The computer backend is a design with an unintegrated prototype.

XI. FUTURE WORK

The next step is the computer backend, in an order that keeps every new action behind a check: the path and process policy and the `action_gate` node first; then a typed post-condition library that subsumes the existing file predicate; then file and process actions within a sandboxed workspace; then an AT-SPI accessibility provider, which can be tested headlessly on Linux, followed by Windows UI Automation and macOS Accessibility; and finally the modality router that lets one task span browser and desktop. In parallel, we will derive per-step postconditions from the planner’s expected outcomes instead of request wording, add a local vision-language judge as a second opinion on the final screenshot, make the evidence log hash-chained, and run the protocol of Section VI, starting with its browser family. Longer term, we plan to separate planning, grounding and verification into cooperating agents, with the verifier given no write access to the environment.

XII. CONCLUSION

Agentic Pilot is a local agent built on the rule that an executed action is not a completed task: it may report completion only when a predicate over the observed state passes, and it keeps the inputs of that decision as evidence. Today it acts in the browser, where testing the completion gate inside the live loop exposed concrete defects; fixing them raised the share of withheld premature completions on our design states from at most 15 to 20 of 23, while held-out states show that rule fixes transfer only partially. We have formulated the same gate for desktop applications, windows, files and processes, where postconditions are typed and exact but actions are often irreversible, and designed a computer backend that pairs those postconditions with deterministic, pre-action permission checks. Implementing that backend and measuring the whole system on real tasks are the work that remains.

ACKNOWLEDGMENT

[Pending] An AI system (Anthropic’s Claude) assisted with analyzing the source code, running the reported component

experiments, searching and checking references, and drafting the text of this paper. The authors reviewed, verified and revised all content and take full responsibility for it.

REFERENCES

- [1] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu *et al.*, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
- [2] C. Zhang, L. Li, S. He, X. Zhang, B. Qiao, S. Qin, M. Ma, Y. Kang, Q. Lin, S. Rajmohan, D. Zhang, and Q. Zhang, “UFO: A UI-focused agent for Windows OS interaction,” in *Proc. 2025 Conf. Nations of the Americas Chapter Assoc. Comput. Linguistics: Human Language Technologies (NAACL-HLT) (Volume 1: Long Papers)*, 2025, pp. 597–622.
- [3] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, “Agent S: An open agentic framework that uses computers like a human,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025.
- [4] H. He, W. Yao, K. Ma, W. Yu, Y. Dai, H. Zhang, Z. Lan, and D. Yu, “WebVoyager: Building an end-to-end web agent with large multimodal models,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 6864–6890.
- [5] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su, “GPT-4V(ision) is a generalist web agent, if grounded,” in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 61 349–61 385.
- [6] C. Zhang, Z. Yang, J. Liu, Y. Li, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: Multimodal agents as smartphone users,” in *Proc. 2025 CHI Conf. Human Factors Comput. Syst. (CHI)*, 2025, pp. 70:1–70:20.
- [7] N. Lukas, A. Salem, R. Sim, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Analyzing leakage of personally identifiable information in language models,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 346–363.
- [8] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [9] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, “Large language models cannot self-correct reasoning yet,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.
- [10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.
- [12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [13] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [14] G. Kim, P. Baldi, and S. McAleer, “Language models can solve computer tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023.
- [15] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Trans. Mach. Learn. Res. (TMLR)*, 2024.
- [16] T. Shi, A. Karpathy, L. Fan, J. Hernandez, and P. Liang, “World of bits: An open-domain platform for web-based agents,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 70, 2017, pp. 3135–3144.
- [17] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2Web: Towards a generalist agent for the web,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.
- [18] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, “Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V,” arXiv:2310.11441, 2023.
- [19] R. Bonatti, D. Zhao, F. Bonacci *et al.*, “Windows agent arena: Evaluating multi-modal OS agents at scale,” arXiv:2409.08264, 2024.
- [20] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024, pp. 100:1–100:13.
- [21] Z. Liu, C. Chen, J. Wang, X. Che, Y. Huang, J. Hu, and Q. Wang, “Fill in the blank: Context-aware automated text input generation for mobile GUI testing,” in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 1355–1367.
- [22] W. Hong, W. Wang, Q. Lv, J. Xu, W. Yu, J. Ji, Y. Wang, Z. Wang, Y. Dong, M. Ding, J. Tang *et al.*, “CogAgent: A visual language model for GUI agents,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024, pp. 14 281–14 290.
- [23] K. Cheng, Q. Sun, Y. Chu, F. Xu, L. YanTao, J. Zhang, and Z. Wu, “SeeClick: Harnessing GUI grounding for advanced visual GUI agents,” in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL) (Volume 1: Long Papers)*, 2024, pp. 9313–9332.
- [24] Y. Qin, Y. Ye, J. Fang *et al.*, “UI-TARS: Pioneering automated GUI interaction with native agents,” arXiv:2501.12326, 2025.
- [25] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, “OmniParser for pure vision based GUI agent,” arXiv:2408.00203, 2024.
- [26] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr, “Autonomous evaluation and refinement of digital agents,” in *Proc. Conf. Language Modeling (COLM)*, 2024.
- [27] M. Sumyk and O. Kosovan, ““are we done yet?”: A vision-based judge for autonomous task completion of computer use agents,” arXiv:2511.20067, 2025.
- [28] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [29] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. 2019 Conf. Empirical Methods Natural Lang. Process. and 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [30] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. 36th Annu. ACM Symp. User Interface Softw. Technol. (UIST)*, 2023.
- [31] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” arXiv:2310.08560, 2023.
- [32] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023.
- [33] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *Proc. 33rd USENIX Secur. Symp. (USENIX Security)*, 2024, pp. 1831–1847.
- [34] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10 471–10 506.
- [35] E. DeBenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, vol. 37, 2024.
- [36] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
- [37] T. Kuntz, A. Duzan, H. Zhao, F. Croce, Z. Kolter, N. Flammarion, and M. Andriushchenko, “OS-Harm: A benchmark for measuring safety of computer use agents,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, 2025.
- [38] E. DeBenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” arXiv:2503.18813, 2025.
- [39] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.
- [40] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai *et al.*, “Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution,” arXiv:2409.12191, 2024.

- [41] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," in *Proc. Mach. Learn. Syst. (MLSys)*, vol. 6, 2024.
- [42] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, p. 186345, 2024.
- [43] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong *et al.*, "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, p. 121101, 2025.